Carrasco Quiñones Diego Alexander

Bitácora de la práctica 08

Saga con compensación y Circuit Breaker

Proyecto: mexi-banco-s08 (microservicios con Docker Compose)

Contenido

- 1. El problema: saldos antes y después de la Parte 0
- 2. La saga: pasos, compensaciones y diagrama de estados
- 3. El código clave explicado
- 4. La compensación funcionando
- 5. Cuando la compensación también falla
- 6. El Circuit Breaker: los tres estados con evidencia
- 7. Lo que la saga no ofrece: el aislamiento

1. El problema: saldos antes y después de la Parte 0

> En una frase: Ana perdió $200 sin que Beto los recibiera porque el cargo y el abono son dos llamadas separadas al servicio cuenta, sin una transacción única ni compensación, y el servicio se detuvo entre las dos.

Paso 1.1. Creación del entorno con Docker

Se levantó todo el sistema con Docker Compose: tres réplicas del servicio cuenta, la base de datos PostgreSQL, y los servicios discovery, gateway, movimiento, notificación, spei y transferencia. Todos aparecieron como healthy.

```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE         CREATED         STATUS                   PORTS
mexi-banco-v06-cuenta-1       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-2       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-3       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-db-1           postgres:17-alpine             "docker-entrypoint.s…"   db              3 minutes ago   Up 3 minutes (healthy)   5432/tcp
mexi-banco-v06-discovery-1    mexi-banco-v06-discovery       "java -jar app.jar"      discovery       3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1      mexi-banco-v06-gateway         "java -jar app.jar"      gateway         3 minutes ago   Up 2 minutes (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-movimiento-1   mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-notificacion-1 mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-spei-1         mexi-banco-v06-spei            "java -jar app.jar"      spei            3 minutes ago   Up 2 minutes (healthy)   8080/tcp
mexi-banco-v06-transferencia-1 mexi-banco-v06-transferencia  "java -jar app.jar"      transferencia   3 minutes ago   Up 2 minutes (healthy)   8080/tcp
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Contenedores del proyecto en ejecución (docker compose ps)

Paso 1.2. Qué hacía el código antes de corregirlo

El método principal de TransferenciaService es transferir(String claveOrigen, String claveDestino, BigDecimal monto). Recibe la CLABE de la cuenta que envía, la CLABE de la cuenta que recibe y la cantidad a transferir. Hacía lo siguiente, en este orden:

1. Comprueba las cuentas: verifica que origen y destino no sean la misma cuenta y consulta ambas.

2. Realiza el cargo: cuentas.cargar(claveOrigen, monto) descuenta el dinero de Ana.

3. Hace una pausa opcional: pausaDeDemo() detiene el proceso un tiempo para poder reproducir el problema.

4. Realiza el abono: cuentas.abonar(claveDestino, monto) intenta agregar el dinero a Beto. Si falla, no hay compensación.

5. Registra la operación: guarda los movimientos, crea el registro de la transferencia y envía las notificaciones.

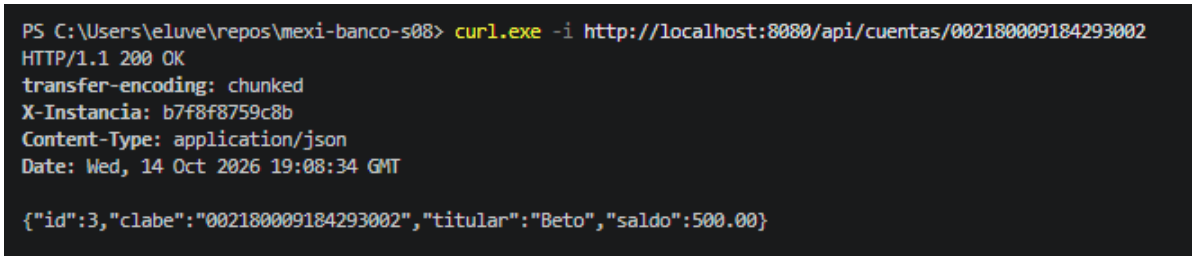
Paso 1.3. Creación de las cuentas y saldos iniciales

Primero se consultaron las cuentas de Ana y de Beto, y después se les abonó el saldo de partida: $1,000.00 a Ana y $500.00 a Beto.

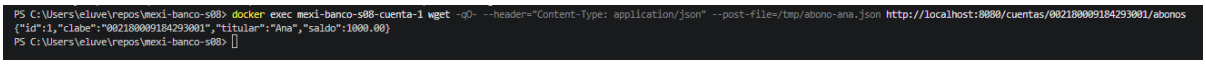
```
PS C:\Users\eluve\repos\mexi-banco-s08> curl.exe -i http://localhost:8080/api/cuentas/002180009184293001
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: b7f8f8759c8b
Content-Type: application/json
Date: Wed, 14 Oct 2026 19:06:52 GMT

{"id":4,"clabe":"002180009184293001","titular":"Ana","saldo":0.00}
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Consulta inicial de la cuenta de Ana, antes de las pruebas (saldo 0.00)



Consulta de la cuenta de Beto



Abono de $1,000.00 a Ana



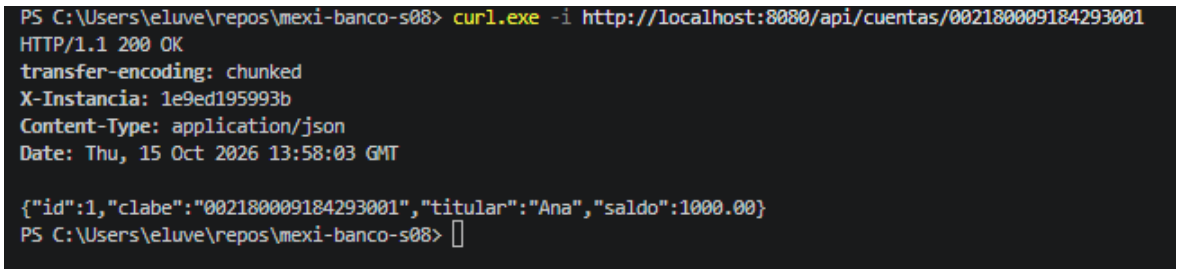
Abono de $500.00 a Beto

Paso 1.4. Verificación de los saldos mediante el Gateway

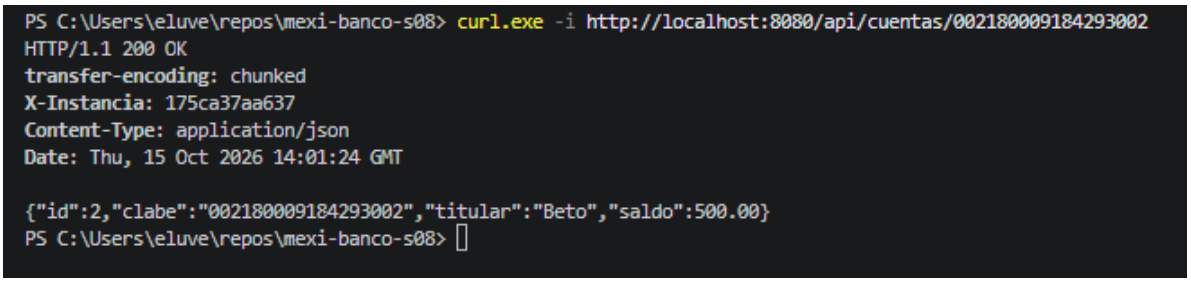
Con curl.exe -i se consultó cada cuenta a través del gateway (puerto 8080). Ana tenía $1,000.00 y Beto $500.00.

```
PS> curl.exe -i http://localhost:8080/api/cuentas/002180009184293001
HTTP/1.1 200 OK
X-Instancia: 1e9ed195993b
{"id":1,"clabe":"002180009184293001","titular":"Ana","saldo":1000.00}

PS> curl.exe -i http://localhost:8080/api/cuentas/002180009184293002
HTTP/1.1 200 OK
X-Instancia: 175ca37aa637
{"id":2,"clabe":"002180009184293002","titular":"Beto","saldo":500.00}
```



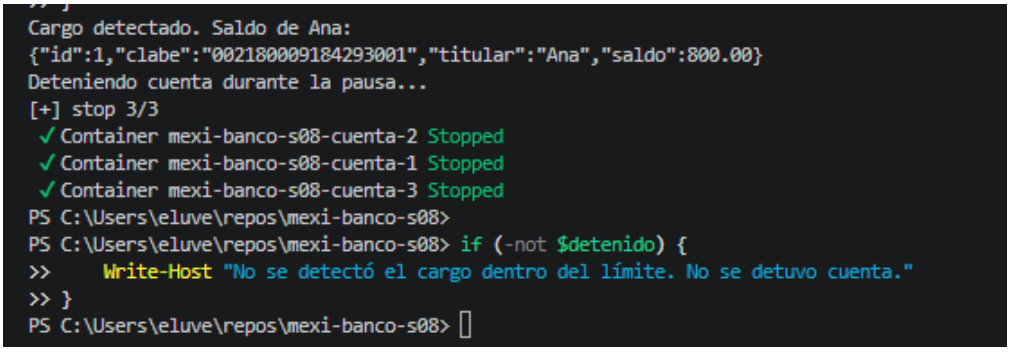
Saldo de Ana vía gateway: $1,000.00



Saldo de Beto vía gateway: $500.00

Paso 1.5. Provocar el fallo: transferencia de $200 con pausa de 20 segundos

Se lanzó una transferencia de $200 de Ana a Beto con una pausa de 20 segundos entre el cargo y el abono. Durante la pausa se detectó que el saldo de Ana ya había bajado a $800.00, y en ese momento se detuvieron las tres réplicas del servicio cuenta.



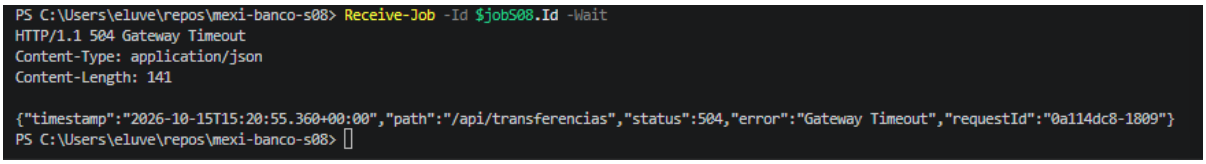
Cargo detectado (Ana con $800.00) y detención de las tres réplicas de cuenta

Esto demuestra que el cargo de $200 ya estaba aplicado y que las réplicas se apagaron durante la pausa, antes de que se pudiera abonar el dinero a Beto.

Paso 1.6. Resultado de la petición: HTTP 504

La transferencia terminó con HTTP 504 Gateway Timeout después de detener las réplicas. Por tanto, el abono a Beto no pudo completarse mientras cuenta estaba detenido.

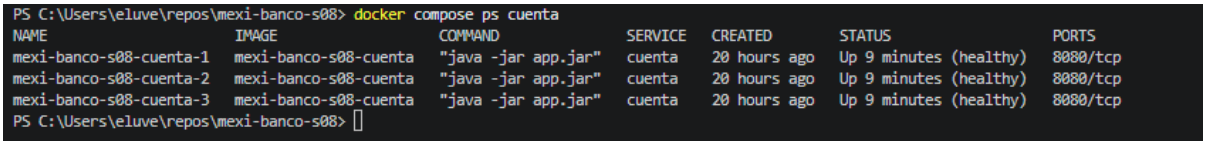
```
PS> Receive-Job -Id $jobS08.Id -Wait
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
Content-Length: 141
{"timestamp":"2026-10-15T15:20:55.360+00:00","path":"/api/transferencias",
 "status":504,"error":"Gateway Timeout","requestId":"0a114dc8-1809"}
```



Respuesta 504 Gateway Timeout

Paso 1.7. Recuperación del servicio y saldos finales

Se volvió a levantar el servicio cuenta (las tres réplicas aparecen healthy) y se consultaron de nuevo los saldos. El cargo de $200 a Ana quedó aplicado incluso después del error 504 y no se revirtió automáticamente, mientras que Beto siguió con $500.



Servicio cuenta recuperado (tres réplicas healthy)

```
PS C:\Users\eluve\repos\mexi-banco-s08> curl.exe -i http://localhost:8080/api/cuentas/002180009184293001
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 42cbefe8f318
Content-Type: application/json
Date: Thu, 15 Oct 2026 16:09:02 GMT

{"id":1,"clabe":"002180009184293001","titular":"Ana","saldo":800.00}
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Ana quedó con $800.00 tras el fallo

```
PS C:\Users\eluve\repos\mexi-banco-s08> curl.exe -i http://localhost:8080/api/cuentas/002180009184293002
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 42cbefe8f318
Content-Type: application/json
Date: Thu, 15 Oct 2026 16:14:43 GMT

{"id":2,"clabe":"002180009184293002","titular":"Beto","saldo":500.00}
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Beto sigue con $500.00

Paso 1.8. Comparación de saldos y resumen

Cuenta	Antes	Después	Diferencia
Ana (origen)	$1,000.00	$800.00	-$200.00
Beto (destino)	$500.00	$500.00	$0.00

En esta parte se prepararon dos cuentas, Ana y Beto, y se hizo una prueba de transferencia de $200. Para reproducir el fallo se configuró una pausa de 20 segundos entre el cargo y el abono. Durante la pausa se vio que Ana ya tenía $800 en lugar de $1,000, y se detuvieron las tres réplicas de cuenta. La petición terminó con un 504. Al reiniciar el servicio, Ana conservaba $800 y Beto seguía con $500: el cargo se aplicó, pero el abono nunca se completó. Los $200 quedaron en un limbo.

Este experimento muestra el problema de los fallos parciales en una arquitectura de microservicios y la necesidad de controlar el estado de las transferencias y de contar con mecanismos de compensación. Esa es la base de la saga que se implementa en la Parte 1.

2. La saga: pasos, compensaciones y diagrama de estados

Paso 2.1. Qué había que corregir

TransferenciaService.java realizaba las operaciones en este orden: (1) descontar el dinero de la cuenta origen, lo cual se confirma en el servicio cuenta; (2) esperar la pausa de demostración, donde se puede detener un servicio para probar fallos; (3) depositar el dinero en la cuenta destino, y si falla el código no devolvía el dinero; (4) registrar movimientos y guardar la transferencia, que ocurre después del abono y por eso un fallo anterior dejaba la operación sin registro completo.

Paso 2.2. Los pasos de la saga y la compensación de cada uno

Paso	Acción	Estado al terminar	Compensación si algo falla
1	Registrar la transferencia	INICIADA	No hace falta: todavía no se movió dinero.
2	Cargar el monto a la cuenta origen (cuentas.cargar)	CARGADA	No aplica; es el paso que se debe deshacer si falla el siguiente.
3	Abonar el monto a la cuenta destino (cuentas.abonar)	ABONADA	Si falla: devolver el dinero a la cuenta origen con cuentas.compensar y una clave única (transferencia-{id}-compensacion).
4	Registrar movimientos y notificar	COMPLETADA	Ya no hay devolución de dinero; el dinero está en el destino.
Compensación	Devolución a origen y movimiento de tipo COMPENSACION	COMPENSADA (o COMPENSACION_PENDIENTE si no se pudo confirmar)	Se reintenta con el endpoint reintentar-compensacion. Es idempotente.

Paso 2.3. Estados de la transferencia

Estado	Significado
INICIADA	La transferencia está registrada, pero todavía no se ha retirado el dinero.
CARGADA	El dinero salió de la cuenta de origen.
ABONADA	El dinero llegó a la cuenta de destino.
COMPLETADA	La transferencia terminó correctamente.
COMPENSADA	Falló el abono y se devolvió el dinero a la cuenta de origen.
COMPENSACION_PENDIENTE	Falló el abono y no se pudo confirmar la devolución; hace falta reintentarla.

Paso 2.4. Diagrama de estados

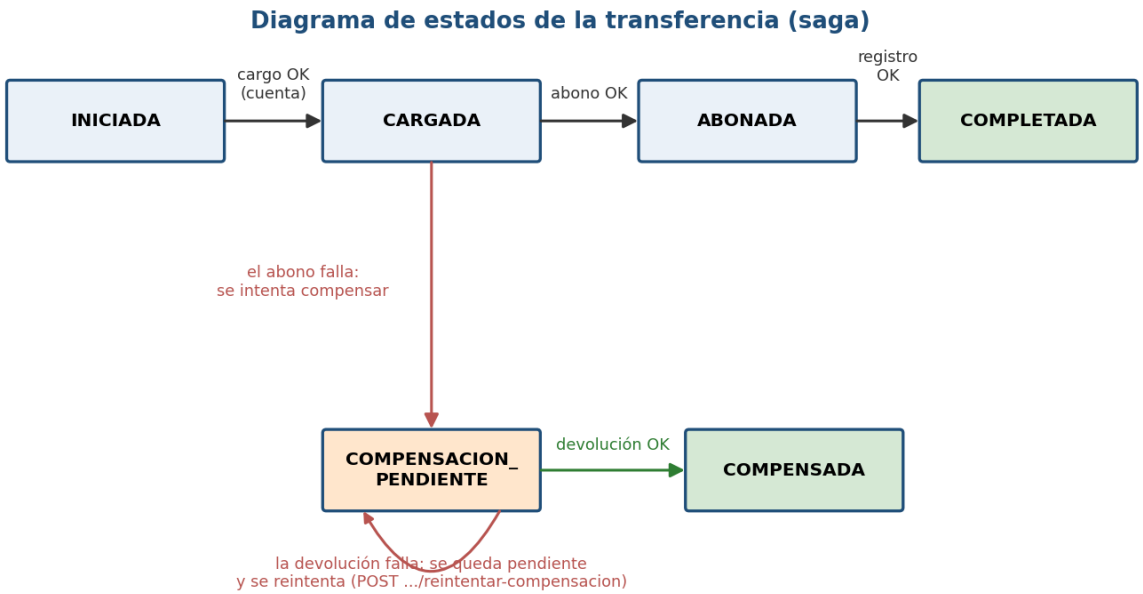


Diagrama de estados de la transferencia

Paso 2.5. Preparación de la compensación en el servicio cuenta

Antes de tocar la transferencia se agregó en el servicio cuenta una operación que devuelve el dinero a la cuenta de origen (POST /cuentas/{clabe}/compensaciones) y que usa una clave de operación única para no duplicar la devolución si se reintenta. Para eso se creó la entidad Compensacion, cuyo campo claveOperacion tiene la restricción unique. Así la base de datos misma impide registrar dos compensaciones con la misma clave.

3. El código clave explicado

Los siguientes fragmentos son los cambios principales, en el orden en que se hicieron. Son ejemplos explicativos de la lógica revisada, no una copia literal de todos los archivos.

3.1. EstadoTransferencia.java: los estados de la saga

```
package mx.mexibanco.transferencia;

public enum EstadoTransferencia {
    INICIADA, CARGADA, ABONADA, COMPLETADA,
    COMPENSADA, COMPENSACION_PENDIENTE
}
```

Es la lista de fases por las que pasa una transferencia. Se creó con un comando de PowerShell (Set-Content) en la carpeta transferencia/src/main/java/mx/mexibanco/transferencia.

3.2. Transferencia.java: guardar el estado

```
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 30)
private EstadoTransferencia estado;

public Transferencia(String claveOrigen, String claveDestino, BigDecimal monto) {
    ...
    this.estado = EstadoTransferencia.INICIADA;
}

public void cambiarEstado(EstadoTransferencia estado) {
    this.estado = estado;
}
```

Se reemplazó la clase conservando los campos actuales (id, claveOrigen, claveDestino, monto, fecha) y se añadió el estado. El campo estado permite guardar en la base de datos la fase actual. @Enumerated(EnumType.STRING) guarda el nombre (por ejemplo CARGADA) en vez de un número. El constructor inicia cada transferencia en INICIADA y cambiarEstado(...) permite avanzar la saga y guardar cada cambio.

3.3. Compensacion.java: evitar devoluciones duplicadas

```
@Entity
public class Compensacion {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String claveOperacion;   // ej. transferencia-1-compensacion

    @Column(nullable = false, length = 18) private String clabe;
    @Column(nullable = false, precision = 19, scale = 2) private BigDecimal monto;
    @Column(nullable = false) private Instant fecha;
}
```

Cada devolución queda registrada con una clave única. Un repositorio permite consultar si una compensación ya existe, y unique = true hace que la base de datos rechace dos registros con la misma clave.

3.4. TransferenciaService.java: orquestar la compensación

```
if (transferencia.getEstado() != EstadoTransferencia.COMPENSACION_PENDIENTE) {
    throw new ResponseStatusException(HttpStatus.UNPROCESSABLE_ENTITY);
}
cuentas.compensar(transferencia.getClaveOrigen(), transferencia.getMonto(),
                  "transferencia-" + id + "-compensacion");
movimientos.registrar(transferencia.getClaveOrigen(), TipoMovimiento.COMPENSACION,
                      transferencia.getMonto(), "Compensacion de transferencia " + id);
transferencia.setEstado(EstadoTransferencia.COMPENSADA);
```

El método reintentarCompensacion(Long id) busca la transferencia y verifica que esté realmente pendiente; si no lo está responde 422. Después pide a cuenta la devolución, registra el movimiento y cambia el estado a COMPENSADA. Si ocurre un error en el proceso, debe conservarse el estado pendiente para permitir otro intento. TransferenciaService funciona como orquestador: controla los pasos y actualiza el estado según el resultado de cada operación.

3.5. CuentaService.java: compensar sin duplicar

```
compensaciones.findByClaveOperacion(claveOperacion);

if (compensacionExistente != null) {
    // Comprobar que corresponda a la misma cuenta y monto.
    return consultar(clabe);
}
// Si no existe, devolver el dinero y registrar la compensación.
```

Antes de devolver el dinero, el servicio comprueba si esa operación ya fue compensada. Si existe el registro, devuelve el saldo actual sin volver a sumar los $100.00. La clave utilizada fue transferencia-1-compensacion. Esto evita duplicar la devolución cuando se repite una solicitud, aunque la clave debe ser única por operación y la comprobación debe protegerse ante solicitudes simultáneas.

3.6. Restricción de la tabla de movimientos

```
ALTER TABLE movimiento DROP CONSTRAINT movimiento_tipo_check;
ALTER TABLE movimiento
ADD CONSTRAINT movimiento_tipo_check
CHECK (tipo IN ('DEPOSITO','TRANSFERENCIA_ENVIADA','TRANSFERENCIA_RECIBIDA',
                'SPEI_ENVIADO','COMPENSACION'));
```

La base de datos rechazaba el tipo COMPENSACION porque movimiento_tipo_check no lo permitía. Al incluirlo, el servicio pudo registrar el movimiento de devolución. Se ejecutó con: docker compose exec db psql -U mexibanco -d movimiento -c "...".

3.7. application.yml del gateway: permitir el endpoint

```
- Path=/api/transferencias,/api/transferencias/{id},/api/transferencias/{id}/reintentar-
compensacion
```

El gateway necesitaba reconocer /api/transferencias/{id}/reintentar-compensacion y dirigir la solicitud al servicio de transferencias. Antes de corregir la ruta la petición respondía 404. Se comprobó el archivo con Select-String (línea 39) y después se aplicó el cambio con docker compose up -d --build gateway.

El código del Circuit Breaker (CuentaCliente.java) se explica en la sección 6.

4. La compensación funcionando

Paso 4.1. Falla provocada del abono

Para fallar el abono a Beto sin apagar ningún servicio se usó la variable de entorno FALLAR_ABONO_CLABE=002180009184293002 (la cuenta de Beto). Antes de la prueba, Ana tenía $800.00 y Beto $500.00.

Paso 4.2. Transferencia de $100 y primer resultado

```
$cuerpo = @{ claveOrigen = "002180009184293001"
             claveDestino = "002180009184293002"
             monto = 100 } | ConvertTo-Json

Invoke-RestMethod -Uri "http://localhost:8080/api/transferencias" -Method Post `
    -ContentType "application/json" -Body $cuerpo

id : 1   monto : 100   estado : COMPENSACION_PENDIENTE
fecha : 2026-10-23T19:11:10.101850315Z
```

La transferencia quedó en COMPENSACION_PENDIENTE y los saldos siguieron en $800.00 para Ana y $500.00 para Beto. El abono falló a propósito y la saga trató de devolver el dinero a Ana.

```
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/transferencias/1" -Method Get


id          : 1
claveOrigen  : 002180009184293001
claveDestino : 002180009184293002
monto        : 100.00
fecha        : 2026-10-23T19:11:10.101850Z
estado       : COMPENSACION_PENDIENTE



PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/002180009184293001" -Method Get

id clabe             titular  saldo
-- -----             -------  -----
 1 002180009184293001 Ana     800.00


PS C:\Users\eluve\repos\mexi-banco-s08>
```

Transferencia 1 en estado COMPENSACION_PENDIENTE y saldo de Ana en $800.00

```
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Uri "http://localhost:8080/api/cuentas/002180009184293002" -Method Get

id clabe             titular  saldo
-- -----             -------  -----
 2 002180009184293002 Beto    500.00
```

Saldo de Beto: $500.00

Con esto se reconstruye lo ocurrido: (1) la transferencia intentó cargar $100 a Ana; (2) el abono a Beto falló, como se había configurado; (3) la saga llamó al servicio cuenta para compensar a Ana y el registro confirma que se atendió POST /cuentas/.../compensaciones; (4) el servicio movimiento intentó guardar el tipo COMPENSACION pero la restricción de la base de datos lo rechazó; (5) la saga terminó en COMPENSACION_PENDIENTE aunque los saldos consultados seguían en $800 y $500.

Para verificar si la compensación de la transferencia 1 ya quedó registrada en la cuenta de Ana se consultó la tabla compensacion: hay un solo registro con clave transferencia-1-compensacion y monto 100.00.

```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM compensacion;"
 id |       clabe        |       clave_operacion       |            fecha            | monto
----+--------------------+-----------------------------+-----------------------------+--------
  1 | 002180009184293001 | transferencia-1-compensacion | 2026-10-23 19:11:12.295244+00 | 100.00
(1 row)

PS C:\Users\eluve\repos\mexi-banco-s08>
```

Registro de la compensación en la tabla compensacion

Paso 4.3. Correcciones

Se corrigió la restricción de la tabla movimiento para permitir COMPENSACION (código 3.6) y se corrigió la ruta del gateway añadiendo el endpoint de reintento (código 3.7). Después de reconstruir el gateway con docker compose up -d --build gateway, las imágenes se construyeron y el contenedor del gateway se inició.

```
[+] up 4/4
 ✓ Image mexi-banco-s08-discovery Built
 ✓ Image mexi-banco-s08-gateway Built
 ✓ Container mexi-banco-s08-discovery-1 Healthy
 ✓ Container mexi-banco-s08-gateway-1 Started
```

Paso 4.4. Reintento de la compensación y estado final

```
Invoke-RestMethod -Method Post -Uri "http://localhost:8080/api/transferencias/1/reintentar-
compensacion"

id : 1   claveOrigen : 002180009184293001   claveDestino : 002180009184293002
monto : 100.00   fecha : 2026-10-23T19:11:10.101850Z
estado : COMPENSADA
```

```
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/transferencias/1" | ConvertTo-Json -Depth 5
{
    "id":  1,
    "claveOrigen":  "002180009184293001",
    "claveDestino":  "002180009184293002",
    "monto":  100.00,
    "fecha":  "2026-10-23T19:11:10.101850Z",
    "estado":  "COMPENSADA"
}
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Estado final de la transferencia 1: COMPENSADA

El problema quedó resuelto y la transferencia pasó a COMPENSADA: en teoría el dinero regresó a la cuenta original. Las cuentas quedaron intactas: Ana con $800.00 y Beto con $500.00.

```
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/cuentas/002180009184293001" | ConvertTo-Json -Depth 5
{
    "id":  1,
    "clabe":  "002180009184293001",
    "titular":  "Ana",
    "saldo":  800.00
}
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/cuentas/002180009184293002" | ConvertTo-Json -Depth 5
{
    "id":  2,
    "clabe":  "002180009184293002",
    "titular":  "Beto",
    "saldo":  500.00
}
```

Saldos finales: Ana $800.00, Beto $500.00

En la base de datos de movimientos se guardó la cuenta de Ana, el monto y la referencia, con tipo COMPENSACION, lo que confirma que se registró la devolución.

```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose exec db psql -U mexibanco -d movimiento -c "SELECT * FROM movimiento WHERE tipo = 'COMPENSACION';"
 id |    clabe_cuenta    |             fecha             | monto |          referencia           | saldo_resultante |    tipo
----+--------------------+-------------------------------+-------+-------------------------------+------------------+-------------
  2 | 002180009184293001 | 2026-10-23 20:00:06.941426+00 |100.00 | Compensacion de transferencia 1 |           800.00 | COMPENSACION
(1 row)

PS C:\Users\eluve\repos\mexi-banco-s08>
```

Movimiento de tipo COMPENSACION guardado en la base de datos

Paso 4.5. Reintento repetido: idempotencia

Se ejecutó otra vez el endpoint de compensación. El servidor respondió HTTP 422 Unprocessable Entity y rechazó el segundo intento. Después se vio que el saldo de Ana seguía en $800, que el total de compensaciones registradas era 1 y que la transferencia continuaba COMPENSADA. Esto demuestra que el segundo intento no duplicó la devolución. La compensación debe ser idempotente para evitar que los reintentos devuelvan el dinero más de una vez, incluso si un servicio falla o una solicitud se procesa repetidamente.

```
PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Post -Uri "http://localhost:8080/api/transferencias/1/reintentar-compensacion"
Invoke-RestMethod : Error en el servidor remoto: (422) Unprocessable Entity.
En línea: 1 Carácter: 1
+ Invoke-RestMethod -Method Post -Uri "http://localhost:8080/api/transf ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/cuentas/002180009184293001"

id clabe              titular  saldo
-- -----              -------  -----
 1 002180009184293001 Ana     800.00


PS C:\Users\eluve\repos\mexi-banco-s08> docker compose exec db psql -U mexibanco -d cuenta -c "SELECT COUNT(*) AS total_compensaciones FROM compensacion WHERE clave_operacion = 'transferencia-1-compensacion';"
 total_compensaciones
----------------------
                    1
(1 row)

PS C:\Users\eluve\repos\mexi-banco-s08> Invoke-RestMethod -Method Get -Uri "http://localhost:8080/api/transferencias/1" | ConvertTo-Json -Depth 5
{
    "id":  1,
    "claveOrigen":  "002180009184293001",
    "claveDestino":  "002180009184293002",
    "monto":  100.00,
    "fecha":  "2026-10-23T19:11:10.101850Z",
    "estado":  "COMPENSADA"
}
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Segundo intento: 422, saldo sin cambio y una sola compensación

Paso 4.6. Resumen de la compensación

TransferenciaService actuó como orquestador. Cuando falló el abono a la cuenta destino, la transferencia quedó en COMPENSACION_PENDIENTE hasta que se pudo reintentar. Después de corregir la ruta del gateway y de permitir el tipo COMPENSACION en la tabla de movimientos, el reintento terminó correctamente y la transferencia pasó a COMPENSADA. Las comprobaciones mostraron que

Ana quedó en $800.00, Beto permaneció en $500.00, se registró un movimiento de compensación por $100.00 y existió un solo registro de compensación para la clave transferencia-1-compensacion.

Momento	Ana	Beto	Estado
Antes de la transferencia	$800.00	$500.00	-
Tras el abono fallido	$800.00	$500.00	COMPENSACION_PENDIENTE
Tras el reintento	$800.00	$500.00	COMPENSADA

5. Cuando la compensación también falla

Si la devolución a Ana también falla, por ejemplo porque el servicio cuenta está apagado en ese momento, la transferencia debe quedar en COMPENSACION_PENDIENTE, en lugar de quedarse indefinidamente en CARGADA. Ese estado deja constancia en la base de datos de que el dinero salió de Ana, no llegó a Beto y todavía hay que devolverlo.

Cómo lo terminaría

- Un reintento programado (por ejemplo con @Scheduled) que cada cierto tiempo busque las transferencias en COMPENSACION_PENDIENTE y vuelva a llamar a cuentas.compensar.
- Usar siempre la misma clave de operación (transferencia-{id}-compensacion) para que el reintento sea idempotente y nunca se devuelva el dinero dos veces.
- Pasar la transferencia a COMPENSADA solo cuando cuenta confirme la devolución y se registre el movimiento.
- Poner un límite de reintentos con esperas crecientes y una alerta para revisión manual si no se logra.
- Proteger la comprobación de duplicados ante solicitudes simultáneas.

> En esta práctica se comprobó el estado pendiente, el reintento exitoso manual y el rechazo de un segundo intento. El reintento programado automático y la prueba con cuenta apagada durante la compensación quedan como aspectos por implementar o verificar.

6. El Circuit Breaker: los tres estados con evidencia

Paso 6.1. Objetivo

Implementar un Circuit Breaker en el microservicio transferencia para controlar las llamadas al microservicio cuenta. Con esto se evita que el sistema siga haciendo peticiones a un servicio que no está disponible y se permite intentar recuperarlo después de un tiempo de espera. Se implementó manualmente, sin Resilience4j, con los estados cerrado, abierto y semiabierto.

Paso 6.2. Código implementado (CuentaCliente.java)

El archivo está en transferencia/src/main/java/mx/mexibanco/clientes/CuentaCliente.java y se reemplazó por completo. Contiene la lógica que controla las llamadas al servicio cuenta.

```
private enum Estado { CERRADO, ABIERTO, SEMIABIERTO }
private Estado estado = Estado.CERRADO;
private final int maximoFallos = 3;        // abre tras tres fallas
private final long esperaMs = 10_000;      // espera 10 s antes de probar
private int fallos = 0;
private long instanteApertura = 0;
private boolean pruebaEnCurso = false;
```

Se definieron tres estados. El inicial es CERRADO, el límite es de tres fallos consecutivos y la espera antes de intentar la recuperación es de 10 segundos. pruebaEnCurso asegura que solo haya una llamada de prueba en ejecución cuando el circuito está semiabierto.

```
private synchronized void permitirLlamada() {
    if (estado == Estado.ABIERTO) {
        long transcurrido = (System.nanoTime() - instanteApertura) / 1_000_000;
        if (transcurrido >= esperaMs) {
            estado = Estado.SEMIABIERTO; pruebaEnCurso = false;
            System.out.println("[CIRCUITO] ABIERTO -> SEMIABIERTO");
        } else {
            System.out.println("[CIRCUITO] Llamada rechazada: circuito ABIERTO");
            throw new ErrorRemotoException("cuenta", 503,
                "Servicio cuenta temporalmente no disponible: circuito abierto");
        }
    }
    if (estado == Estado.SEMIABIERTO) {
        if (pruebaEnCurso) throw new ErrorRemotoException("cuenta", 503,
                "Servicio cuenta en prueba de recuperacion");
        pruebaEnCurso = true;
        System.out.println("[CIRCUITO] Enviando llamada de prueba");
    }
}
```

permitirLlamada() decide si se puede llamar al servicio remoto. Con el circuito abierto rechaza las peticiones hasta que pasen los 10 segundos; después cambia a semiabierto y deja pasar una llamada de prueba. synchronized protege los cambios de estado ante llamadas simultáneas y System.nanoTime() mide la espera sin depender de cambios en el reloj del sistema.

```
private synchronized void registrarExito() {
    if (estado == Estado.SEMIABIERTO) {
```

```
        estado = Estado.CERRADO; fallos = 0; pruebaEnCurso = false;
        System.out.println("[CIRCUITO] SEMIABIERTO -> CERRADO");
    } else if (estado == Estado.CERRADO) { fallos = 0; }
}

private synchronized void registrarFallo() {
    fallos++;
    System.out.println("[CIRCUITO] Falla consecutiva: " + fallos);
    if (estado == Estado.SEMIABIERTO || fallos >= maximoFallos) {
        estado = Estado.ABIERTO; instanteApertura = System.nanoTime();
        pruebaEnCurso = false;
        System.out.println("[CIRCUITO] Circuito ABIERTO");
    }
}
```

registrarExito() cierra el circuito después de una prueba exitosa y reinicia el contador. registrarFallo() cuenta fallas y abre el circuito al llegar a tres; si la llamada de prueba falla en semiabierto, también lo vuelve a abrir y reinicia la espera.

```
private <T> T ejecutar(java.util.function.Supplier<T> llamada) {
    permitirLlamada();
    try {
        T resultado = llamada.get();
        registrarExito();
        return resultado;
    } catch (ErrorRemotoException error) {
        if (error.getEstado() >= 500) registrarFallo(); else registrarExito();
        throw error;
    } catch (RestClientException error) {   // incluye timeouts
        registrarFallo();
        throw error;
    }
}
```

ejecutar() concentra el control: verifica el circuito, hace la llamada y registra éxito o fallo. Las operaciones consultar, cargar, abonar y compensar usan este mismo método, así que no se repite la lógica. Los errores HTTP 4xx no cuentan como fallas del circuito (por ejemplo saldo insuficiente o cuenta inexistente), pero sí se devuelven al cliente; las respuestas 5xx y los errores de comunicación sí cuentan.

Paso 6.3. Compilación e inicio

Comando	Resultado
docker compose build transferencia	BUILD SUCCESS; se construyó la imagen mexi-banco-s08-transferencia (sin errores de compilación).
docker compose up -d transferencia	El servicio inició en segundo plano, Spring Boot terminó de arrancar y se registró en Eureka como TRANSFERENCIA con estado UP.
docker compose ps transferencia	Contenedor en ejecución con health: starting al principio.
docker compose ps -a cuenta	Las tres instancias de cuenta aparecieron detenidas (Exited 137), por lo que no estaban disponibles para las pruebas.
docker compose logs --since=3m transferencia	Inicio correcto (Started TransferenciaApplication) y registro en Eureka, sin errores.

Paso 6.4. Estado CERRADO que se abre: primeras fallas

Con el servicio cuenta apagado se hizo una primera prueba con CLABEs ficticias (PRUEBA000000000001 y PRUEBA000000000002) y monto 1, para no guardar transferencias ni mover dinero. La petición devolvió HTTP 500 porque el balanceador de Eureka no encontró instancias de cuenta. El código del circuito capturó la excepción como fallo de comunicación y el contador subió a 1. Esto demuestra que registró el primer fallo, pero todavía no que se haya abierto.

```
PS C:\Users\eluve\repos\mexi-banco-s08> try {
>>     Invoke-RestMethod `
>>         -Method Post `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -ContentType "application/json" `
>>         -Body '{"claveOrigen":"PRUEBA000000000001","claveDestino":"PRUEBA000000000002","monto":1}'
>> }
>> catch {
>>     Write-Host "Resultado de la prueba:"
>>     Write-Host $_.Exception.Message
>>     if ($_.Exception.Response) {
>>         Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     }
>> }
Resultado de la prueba:
Error en el servidor remoto: (500) Error interno del servidor.
Código HTTP: 500
```

Primera prueba: HTTP 500 y primer fallo registrado

Se repitieron dos pruebas más (2 y 3) con las mismas CLABEs. Si la comunicación con cuenta falla igual, transferir() se detiene en la primera consulta, antes de guardar la transferencia o ejecutar cargos y abonos.

```
transferencia-1  |
PS C:\Users\eluve\repos\mexi-banco-s08> for ($i = 2; $i -le 3; $i++) {
>>     Write-Host "`nPrueba $i"
>>
>>     try {
>>         Invoke-RestMethod `
>>             -Method Post `
>>             -Uri "http://localhost:8080/api/transferencias" `
>>             -ContentType "application/json" `
>>             -Body '{"claveOrigen":"PRUEBA000000000001","claveDestino":"PRUEBA000000000002","monto":1}'
>>     }
>>     catch {
>>         Write-Host "Respuesta:" $_.Exception.Message
>>     }
>> }

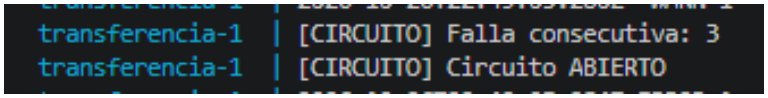
Prueba 2
Respuesta: Error en el servidor remoto: (500) Error interno del servidor.

Prueba 3
Respuesta: Error en el servidor remoto: (500) Error interno del servidor.
```

Pruebas 2 y 3: respuesta 500 en ambas

```
transferencia-1  | 2026-10-26T22:49:05.000Z  WARN 1 -
transferencia-1  | [CIRCUITO] Falla consecutiva: 2
```

Registro: falla consecutiva 2

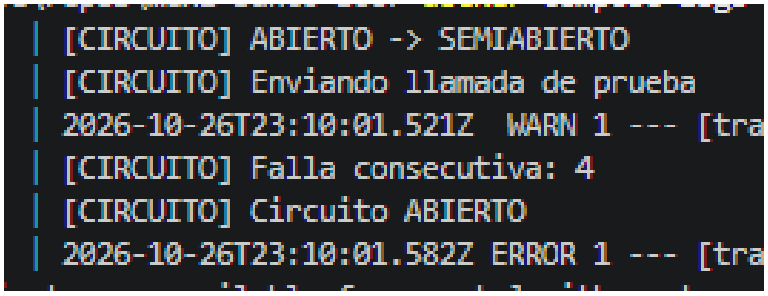


Registro: falla consecutiva 3 y circuito ABIERTO

Con esto se comprueba que el Circuit Breaker llega al límite de tres fallos consecutivos y cambia al estado ABIERTO.

Paso 6.5. Estado ABIERTO a SEMIABIERTO y de vuelta a ABIERTO

Se envió una solicitud adicional con el servicio cuenta todavía apagado, para ver si el circuito abierto impedía la llamada remota. Los registros muestran que el circuito pasó de ABIERTO a SEMIABIERTO (ya habían pasado más de 10 segundos), envió la llamada de prueba, falló (falla consecutiva 4) y se volvió a abrir.



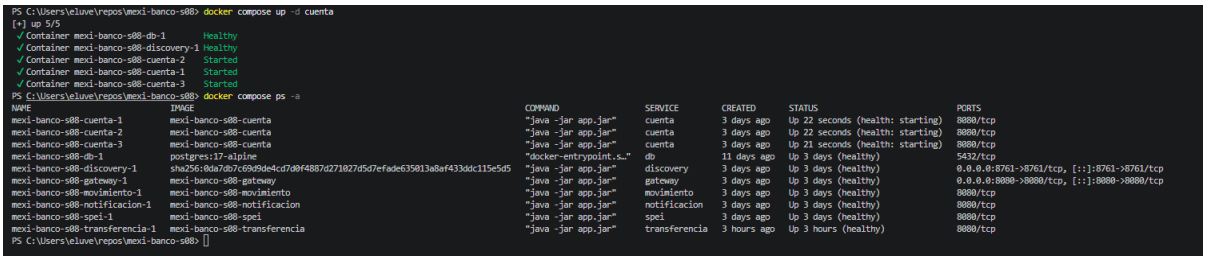
ABIERTO -> SEMIABIERTO, llamada de prueba fallida y circuito ABIERTO otra vez

```
PS C:\Users\eluve\repos\mexi-banco-s08> try {
>>     Invoke-RestMethod `
>>         -Method Post `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -ContentType "application/json" `
>>         -Body '{"claveOrigen":"PRUEBA000000000001","claveDestino":"PRUEBA000000000002","monto":1}'
>> }
>> catch {
>>     Write-Host "Respuesta:" $_.Exception.Message
>>     if ($_.Exception.Response) {
>>         Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     }
>> }
Respuesta: Error en el servidor remoto: (500) Error interno del servidor.
Código HTTP: 500
```

La solicitud devolvió HTTP 500, no 503, y sí intentó contactar a cuenta

Paso 6.6. Recuperación: SEMIABIERTO a CERRADO

Para comprobar la recuperación se levantaron de nuevo los contenedores de cuenta, de modo que el circuito se probara con el servicio disponible, sin cuentas reales ni movimientos de dinero.



```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose up -d cuenta
[+] up 5/5
 ✓ Container mexi-banco-s08-db-1          Healthy
 ✓ Container mexi-banco-s08-discovery-1   Healthy
 ✓ Container mexi-banco-s08-cuenta-2      Started
 ✓ Container mexi-banco-s08-cuenta-1      Started
 ✓ Container mexi-banco-s08-cuenta-3      Started
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose ps -a
NAME                          IMAGE                                                                                COMMAND                  SERVICE         CREATED       STATUS                             PORTS
mexi-banco-s08-cuenta-1       mexi-banco-s08-cuenta                                                                "java -jar app.jar"      cuenta          3 days ago    Up 22 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-2       mexi-banco-s08-cuenta                                                                "java -jar app.jar"      cuenta          3 days ago    Up 22 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-3       mexi-banco-s08-cuenta                                                                "java -jar app.jar"      cuenta          3 days ago    Up 21 seconds (health: starting)   8080/tcp
mexi-banco-s08-db-1           postgres:17-alpine                                                                   "docker-entrypoint.s…"   db              11 days ago   Up 3 days (healthy)                5432/tcp
mexi-banco-s08-discovery-1    sha256:0da7db7c69d9de4cd7d0f4887d271027d5d7efade635013a8af433ddc115e5d5              "java -jar app.jar"      discovery       3 days ago    Up 3 days (healthy)                0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-s08-gateway-1      mexi-banco-s08-gateway                                                               "java -jar app.jar"      gateway         3 days ago    Up 3 days (healthy)                0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-s08-movimiento-1   mexi-banco-s08-movimiento                                                            "java -jar app.jar"      movimiento      3 days ago    Up 3 days (healthy)                8080/tcp
mexi-banco-s08-notificacion-1 mexi-banco-s08-notificacion                                                          "java -jar app.jar"      notificacion    3 days ago    Up 3 days (healthy)                8080/tcp
mexi-banco-s08-spei-1         mexi-banco-s08-spei                                                                  "java -jar app.jar"      spei            3 days ago    Up 3 days (healthy)                8080/tcp
mexi-banco-s08-transferencia-1 mexi-banco-s08-transferencia                                                        "java -jar app.jar"      transferencia   3 hours ago   Up 3 hours (healthy)               8080/tcp
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Servicio cuenta iniciándose de nuevo

Se repitió la solicitud con las CLABEs ficticias. Respondió HTTP 404 porque la cuenta no existe. Ese 404 no significa que el Circuit Breaker haya fallado: según la lógica implementada, una respuesta menor a 500 no se considera falla del circuito. Así se distingue un problema de comunicación o del servidor de un error de negocio. Tampoco demuestra que se haya hecho una transferencia, porque la solicitud no encontró el recurso.

```
PS C:\Users\eluve\repos\mexi-banco-s08> try {
>>     Invoke-RestMethod `
>>         -Method Post `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -ContentType "application/json" `
>>         -Body '{"claveOrigen":"PRUEBA000000000001","claveDestino":"PRUEBA000000000002","monto":1}'
>> }
>> catch {
>>     Write-Host "Respuesta:" $_.Exception.Message
>>     if ($_.Exception.Response) {
>>         Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     }
>> }
Respuesta: Error en el servidor remoto: (404) No se encontró.
Código HTTP: 404
```

Solicitud tras la recuperación: HTTP 404 (cuenta ficticia inexistente)

```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose logs --since=3m transferencia
transferencia-1  | 2026-10-26T23:25:17.717Z  INFO 1 --- [transferencia] [rap-executor-%d] c.n.d.s.r.aws.ConfigClusterResolver      : Resolving eureka endpoints via configuration
transferencia-1  | [CIRCUITO] ABIERTO -> SEMIABIERTO
transferencia-1  | [CIRCUITO] Enviando llamada de prueba
transferencia-1  | [CIRCUITO] SEMIABIERTO -> CERRADO
PS C:\Users\eluve\repos\mexi-banco-s08>
```

Registros del circuito: ABIERTO -> SEMIABIERTO -> CERRADO

Paso 6.7. Confirmación final

```
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose up -d cuenta
[+] up 5/5
 ✓ Container mexi-banco-s08-db-1         Healthy
 ✓ Container mexi-banco-s08-discovery-1  Healthy
 ✓ Container mexi-banco-s08-cuenta-2     Started
 ✓ Container mexi-banco-s08-cuenta-1     Started
 ✓ Container mexi-banco-s08-cuenta-3     Started
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose ps -a
NAME                        IMAGE                       COMMAND                  SERVICE   CREATED      STATUS                          PORTS
mexi-banco-s08-cuenta-1     mexi-banco-s08-cuenta       "java -jar app.jar"      cuenta    3 days ago   Up 7 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-2     mexi-banco-s08-cuenta       "java -jar app.jar"      cuenta    3 days ago   Up 8 seconds (health: starting)   8080/tcp
mexi-banco-s08-cuenta-3     mexi-banco-s08-cuenta       "java -jar app.jar"      cuenta    3 days ago   Up 7 seconds (health: starting)   8080/tcp
```

Servicio cuenta levantado de nuevo (compose up y compose ps)

```
PS C:\Users\eluve\repos\mexi-banco-s08> try {
>>     Invoke-RestMethod `
>>         -Method Post `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -ContentType "application/json" `
>>         -Body '{"claveOrigen":"PRUEBA000000000001","claveDestino":"PRUEBA000000000002","monto":1}'
>> }
>> catch {
>>     Write-Host "Respuesta:" $_.Exception.Message
>>
>>     if ($_.Exception.Response) {
>>         Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     }
>> }
Respuesta: Error en el servidor remoto: (404) No se encontró.
Código HTTP: 404
PS C:\Users\eluve\repos\mexi-banco-s08> docker compose logs --since=3m transferencia | Select-String "\[CIRCUITO\]"

transferencia-1  | [CIRCUITO] ABIERTO -> SEMIABIERTO
transferencia-1  | [CIRCUITO] Enviando llamada de prueba
transferencia-1  | [CIRCUITO] SEMIABIERTO -> CERRADO
```

Respuesta 404 y filtrado de los registros [CIRCUITO]

Esto confirma que el circuito esperó el tiempo configurado, permitió una llamada de prueba y volvió al estado cerrado. La respuesta 404 se debe a las CLABEs ficticias: la llamada llegó a un error de negocio, y como el código trata los 4xx como éxito para el circuito, este se cerró aunque la transferencia de prueba no se realizó.

Paso 6.8. Resumen de los tres estados

Estado	Qué hace	Evidencia
CERRADO	Deja pasar las llamadas y cuenta las fallas consecutivas.	Fallas consecutivas 1, 2 y 3; después de recuperarse, SEMIABIERTO -> CERRADO.
ABIERTO	Tras 3 fallas rechaza llamadas (503) durante 10 segundos.	Registro: Falla consecutiva 3 y Circuito ABIERTO; en la falla 4 vuelve a abrirse.
SEMIABIERTO	Pasados los 10 s deja una sola llamada de prueba: si funciona cierra, si falla reabre.	Registros: ABIERTO -> SEMIABIERTO, Enviando llamada de prueba, y SEMIABIERTO -> CERRADO.

Paso 6.9. Observaciones

- El Circuit Breaker centraliza el control de las llamadas entre microservicios y evita enviar peticiones cuando el servicio remoto falla de forma consecutiva.
- Compilar e iniciar el servicio no basta para demostrar que el circuito funciona: hay que hacer pruebas controladas y revisar los registros de cada cambio de estado.
- Los errores de comunicación y las respuestas 5xx cuentan como fallas; los 4xx no, para distinguir problemas de disponibilidad de errores de la solicitud.
- Como cargo, abono y compensación modifican información bancaria, las pruebas se hicieron con CLABEs ficticias para no mover dinero.
- En la prueba con el circuito ya abierto la solicitud devolvió 500 y no un 503 controlado, porque ya habían pasado más de 10 segundos y el circuito estaba semiabierto. Todavía falta evidenciar con una solicitud inmediata el rechazo directo con el mensaje de circuito abierto.

7. Lo que la saga no ofrece: el aislamiento

Una saga garantiza que la operación termine o se compense, pero no es una transacción ACID: le falta la I de aislamiento. Cada paso (cargo, abono, compensación) se confirma de forma local y definitiva en cuenta antes de que termine toda la transferencia, así que sus efectos intermedios son visibles para cualquiera que consulte.

Esto fue lo que pasó en la Parte 0: entre el cargo y el abono, el saldo de Ana ya marcaba $800.00 aunque la transferencia no había terminado y Beto seguía con $500.00. En el caso de la compensación ocurre igual. Alguien que consulte el saldo de la cuenta de origen entre el cargo y la compensación ve un saldo reducido, por ejemplo $700.00 en lugar de $800.00 si la transferencia de $100 todavía no se compensa, y esa lectura es sucia porque corresponde a un estado que luego se deshace. Durante esa ventana el dinero no aparece ni en el origen ni en el destino.

Las consecuencias pueden ser reales: un cliente puede creer que le falta dinero, otra operación puede ser rechazada por saldo insuficiente aunque el dinero vaya a regresar, o se pueden tomar decisiones con un saldo que no es el definitivo. Para reducir el problema se podrían usar bloqueos semánticos (por ejemplo, marcar la transferencia o la cuenta como en proceso), mostrar saldos retenidos o pendientes, ordenar los pasos para que el menos reversible ocurra al final, o consultar siempre el estado de la transferencia junto con el saldo.